```
$ ssmgate db --env dev
```

ssmgate

Acceso seguro a servidores y bases de datos privadas de AWS. Sin SSH, sin puertos abiertos.

Trabajo Fin de Máster · Máster en Programación con IA

Daniel Passarelli · Octubre 2026

EL PROBLEMA

Llegar a una base de datos privada suele pasar por un bastion SSH

Puerto 22 abierto

Un servidor expuesto a internet es superficie de ataque permanente.

Claves compartidas

Claves SSH repartidas por el equipo, difíciles de rotar y de revocar.

Poca trazabilidad

¿Quién entró, cuándo y con qué permisos? Un problema para auditorías.

LA SOLUCIÓN

AWS SSM Session Manager, reducido a un solo comando

```
$ ssmgate db --env dev
```

login SSO → rol → túnel SSM → token IAM 15 min → psql

Conexión **saliente** desde la instancia: cero puertos de entrada. Cada sesión ligada a tu identidad SSO y registrada en AWS.

QUÉ HACE

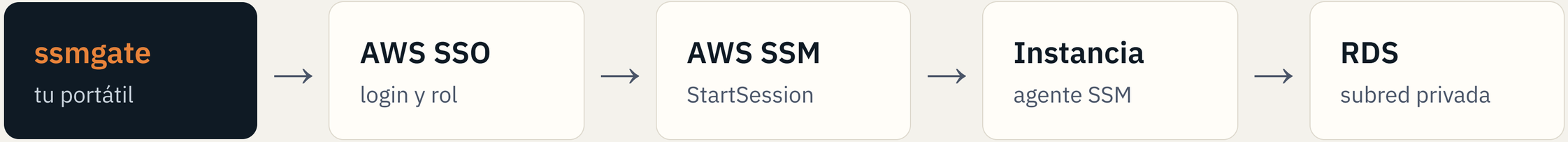
Tres comandos principales

`ssmgate shell bastion`	Shell interactiva en cualquier instancia gestionada por SSM, con un rol SSO por host
`ssmgate db`	Túnel a la base de datos privada + token IAM de 15 minutos + psql con SSL
`ssmgate tunnel`	Túnel abierto para DBeaver o TablePlus; el token se copia al portapapeles

Y además: **envs**, **config init · validate**, **doctor** y el modo **--demo**. Todos exigen **--env** explícito.

CÓMO FUNCIONA

El recorrido de ssmgate db



Login sin navegador

OIDC device flow: URL y código, funciona en WSL o por SSH.

Túnel local

session-manager-plugin oficial reenvía localhost:55432 a la BD.

Sin contraseñas

Token IAM de 15 min firmado para el endpoint real, SSL verify-ca.

GENÉRICO

Tu infraestructura vive en un YAML, no en el código

- Entornos, cuentas, hosts, roles y BBDD
- SSO integrado o perfil de la AWS CLI
- Validación estricta con todos los errores a la vez
- **confirm: true** protege producción

```yaml
environments:
  prod:
    region: us-east-1
    account_id: "222222222222"
    confirm: true
    hosts:
      bastion:
        instance_id: i-0aaa…
        role: BastionSessionOperator
    db:
      endpoint: prod-db.…rds.amazonaws.com
      name: appdb
      local_port: 55433
      via: bastion
      role: RDSOperator
```

FUNCIONALIDADES

Pensado para el día a día de un equipo

Rol por host

Mínimo privilegio: cada servidor exige su propio rol SSO.

Prod protegido

Hay que escribir el nombre del entorno antes de actuar.

Túneles en paralelo

Un puerto local por entorno: dev y prod a la vez.

Sesión reutilizable

Caché SSO compatible con la AWS CLI.

Multiplataforma

Windows, Linux, macOS y WSL; amd64 y arm64.

doctor

Diagnostica config, plugin y psql en un comando.

STACK TECNOLÓGICO

Un binario estático, sin dependencias

Área	Tecnología
Lenguaje	Go 1.26 · CGO_ENABLED=0
CLI y config	spf13/cobra · YAML (yaml.v3, claves estrictas)
AWS	SDK for Go v2: SSO OIDC, SSO, STS, SSM, RDS auth
Canal SSM	session-manager-plugin oficial de AWS
Calidad	go test · golangci-lint v2 · gofmt
CI/CD	GitHub Actions · GoReleaser → GitHub Releases
Desarrollo	Claude Code como asistente de programación

CALIDAD

Probado sin depender de AWS

38
tests unitarios y end-to-end

3
sistemas operativos en CI

6
binarios por release

0
avisos de golangci-lint

La interfaz **Connector** separa la CLI del backend: los tests recorren el árbol de comandos real con el conector demo, sin red.

DESARROLLO CON IA

Claude Code como compañero de programación

Lo que delegué

- Port del script bash a Go
- Refactor a configuración genérica
- Modo demo, tests y CI/CD
- Documentación

Cómo lo verifiqué

- vet, lint y tests en cada cambio
- Pruebas manuales de cada comando
- Revisión del diff y las decisiones de seguridad
- Smoke test del modo demo en cada push

Lo que aprendí

- El contexto del dominio evita "arreglos" erróneos
- Pedir verificación ejecutable
- Diseñar para testear

PRUÉBALO EN UN MINUTO

Modo --demo: sin cuenta de AWS

```
$ ssmgate --demo envs
$ ssmgate --demo shell bastion --env dev
$ ssmgate --demo db --env dev
$ ssmgate --demo tunnel --env prod
```

Mismos pasos que el uso real, un puerto TCP real en localhost y una shell y un psql simulados.

RETOS Y SIGUIENTES PASOS

Lo que costó y lo que viene

Retos técnicos

- El protocolo SSM no está en el SDK: hubo que integrar el plugin
- SSL a través de un túnel: verify-ca
- Usuario de BD distinto del usuario SSO
- Login sin navegador en WSL

Próximos pasos

- Embeber el plugin para un binario 100% autónomo
- Soporte para MySQL / Aurora MySQL
- Distribución por Homebrew y Scoop
- Descubrir instancias por etiquetas

Gracias

`github.com/dapasa/ssmgate`

Binarios: github.com/dapasa/ssmgate/releases

Pruébalo: ssmgate --demo envs

Daniel Passarelli · Máster en Programación con IA